

Configuration Online Service — Modules & MCP Server

(Associate Project)

1. BACKGROUND

The ATLAS DAQ Configuration Online Service manages hardware, software, and detector setup through the OKS (Object Kernel Support) framework. OKS stores configuration as objects that are also linked to each other. Every configuration is a specific git revision, and past run configurations can be looked up by run number via git tags. All modules work only on archived (past) OKS revisions, accessed by run number or tag. No live/production access is needed. This project builds the pieces needed to let an LLM/shifter ask questions about the Configuration service in plain English, and wraps them into a working MCP server.

2. ACCESS LAYER DECISION

Before building the modules that talk to OKS directly, the team should decide how: either through `webdaq` (an existing HTTP/JSON API covering OKS), or through native C++ bindings (via pybind11 or SWIG) around the `oks` library itself. A short comparison/spike is worth doing early, since this choice underlies M4, M5, M6, and M7.

3. MODULES

M1. Run/Revision Resolver

Takes a run number and finds the exact git revision that run's configuration lived in. Runs first, since every other module needs a resolved revision to work on.

M2. Schema Discovery

OKS has a lot of classes and attributes, too many to give an LLM all at once. This module finds just the part of the schema relevant to a given question.

M3. Text-to-OksQuery Translation

Turns a plain-English question into a valid `OksQuery` string, using the schema piece from M2, a set of few-shot examples, and a validate/repair loop that catches bad queries before running them.

M4. Query Execution & Formatting

Runs the query safely (read-only) and turns the raw result into something clean and easy to read, instead of a large raw dump.

M5. Object Lookup

Fetches an object's attributes directly when its exact ID is already known. No translation or query is needed, since there is nothing to interpret.

M6. Relationship Navigation

Handles traversal, connection-type questions, not filter questions. "What computer does this application run on?" isn't something you search for, it's something you follow: start at the application, follow its `RunsOn` link, and you land on the computer. M6 does exactly that: given a starting object and a relationship to follow (or a short chain of them, for multi-step questions), it walks the link and returns what's on the other end.

M7. Revision Diff/Comparison

Compares two run numbers and reports what changed between them: what was added, removed, or modified.

M8. MCP Server Shell

Connects M1 through M7 together as tools an LLM can call by name. No new logic here, just wiring everything into one working agent.

4. EVALUATION & DELIVERABLES

Most modules just need to work correctly: code and tests, nothing more. Two need proper evaluation, because they involve judgment calls rather than fixed answers.

System Architecture & Design Document

A concise document describing the access layer decision, how the modules (M1–M8) fit together, and the end-to-end query execution flow.

Modules (M1–M8)

Source code for all modules, with unit tests.

Evaluation Dataset (shared by M2 and M3)

A set of sample shifter questions paired with the correct `oksQuery` answer, stratified into easy, medium, and hard. Built against existing OKS sample/test data where available, and real scenarios where possible. This dataset is the single source for both evaluations below: M3's accuracy is measured against it directly, and M2's gold set is derived automatically by parsing these same ground-truth queries.

M2. Schema Retrieval

Precision/recall of retrieved schema elements against the gold set. The gold set is derived automatically by parsing the ground-truth queries and extracting the classes, attributes, and relationships they reference, requiring no separate annotation effort.

M3. Translation

- Accuracy is measured by executing the generated query and comparing results to the ground-truth query's results, not by comparing query text.
- Valid-syntax rate is reported separately.
- Failures are broken down by an error taxonomy (e.g. wrong syntax vs. wrong class/attribute selected) rather than a single pass/fail number.
- An ablation is also reported, showing accuracy at each stage: plain prompt only → + few-shot examples → + schema retrieval → + validate/repair loop (full system). This shows what each part of the design contributes.

Whole System

A batch of real questions run end to end through M1–M8, reporting how many get the right final answer, along with reliability, robustness, output consistency, response latency, and resource usage (CPU/memory) for the system as a whole.